

-  Secrets
-  ABAP
-  Ansible
-  Apex
-  AzureResourceManager
-  C
-  C#
-  C++
-  CloudFormation
-  COBOL
-  CSS
-  **Dart**
-  Docker
-  Flex
-  Go
-  HTML
-  Java
-  JavaScript
-  JCL
-  Kotlin
-  Kubernetes
-  Objective C
-  PHP
-  PL/I
-  PL/SQL
-  Python
-  RPG
-  Ruby
-  Scala
-  Swift
-  Terraform
-  Text
-  TypeScript
-  T-SQL
-  VB.NET
-  VB6
-  XML



Dart static code analysis

Unique rules to write Clean DART Code

All rules **115**

 Bug **15**

 Code Smell **100**

Tags ▾

Impact ▾

Clean code attribute ▾

Search by name... 

Empty statements should be removed

Class names should comply with a naming convention

Track uses of "TODO" tags

Deprecated code should be removed

Cyclomatic Complexity of functions should not be too high

Constant names should comply with a naming convention

Unnecessary nullable in final declaration should be removed

Dependencies should be sorted

Unused assignments should be removed

Standard outputs should not be used directly to log anything

Variables should not be initialized with "null"


Empty statements should be removed

Analyze your code

Intentionality - Clear

Maintainability ▾

 Code Smell

 Minor 

 unused

Why is this an issue?

More Info

Empty statements represented by a semicolon `;` are statements that do not perform any operation. They are often the result of a typo or a misunderstanding of the language syntax. It is a good practice to remove empty statements since they don't add value and lead to confusion and errors.

Code examples

Noncompliant code example

```
void doSomething() {  
    ; // Noncompliant - was used as a kind of TODO marker  
}  
  
void f() {  
    if (complicated.expression.foo()); // Noncompliant - the condition doesn't apply to bar  
    bar();  
}  
  
void f() {  
    if (complicated.expression.foo())  
        bar();  
    else ; // Noncompliant else is empty  
        buzz();  
}
```

Compliant solution

```
void doSomething() {  
}  
  
void f() {  
    if (complicated.expression.foo()) {  
        bar();  
    }  
}  
  
void f() {  
    if (complicated.expression.foo())  
        bar();  
    else  
        buzz();  
}
```

Available In:
sonarcloud 

